Producer_Consumer

Assignment: Producer-Consumer Problem

Description: Uses mutex and semaphores to synchronize threads

Producer-Consumer System with Buffer Synchronization

Synchronization Approach

On this assignment, a producer-consumer system was implemented using POSIX threads (pthreads) with a shared circular buffer of size 5. The main goal was to ensure safe concurrent access to shared data avoiding race conditions.

Two synchronization mechanisms were used:

Mutex (Mutual Exclusion Lock)

A mutex is basically a lock that we set(lock) before accessing a shared resource and release(unlock) when we are done. While other threads that try to set, it blocks until we release it. If mutex is removed, two producers could call buffer_inser () at the same time and both would read tail and would write to buffer and increment count.

However, since these operations are not atomic, they result in race conditions causing the result to not be what is expected since they will be accessing the shared memory at the same time and one increment might be lost.

Semaphores

The two semaphores were used and they both work differently; the empty prevents producers from overfilling the buffer and the other semaphore full prevents consumers from underflowing.

Producer_Consumer

Execution Flow:

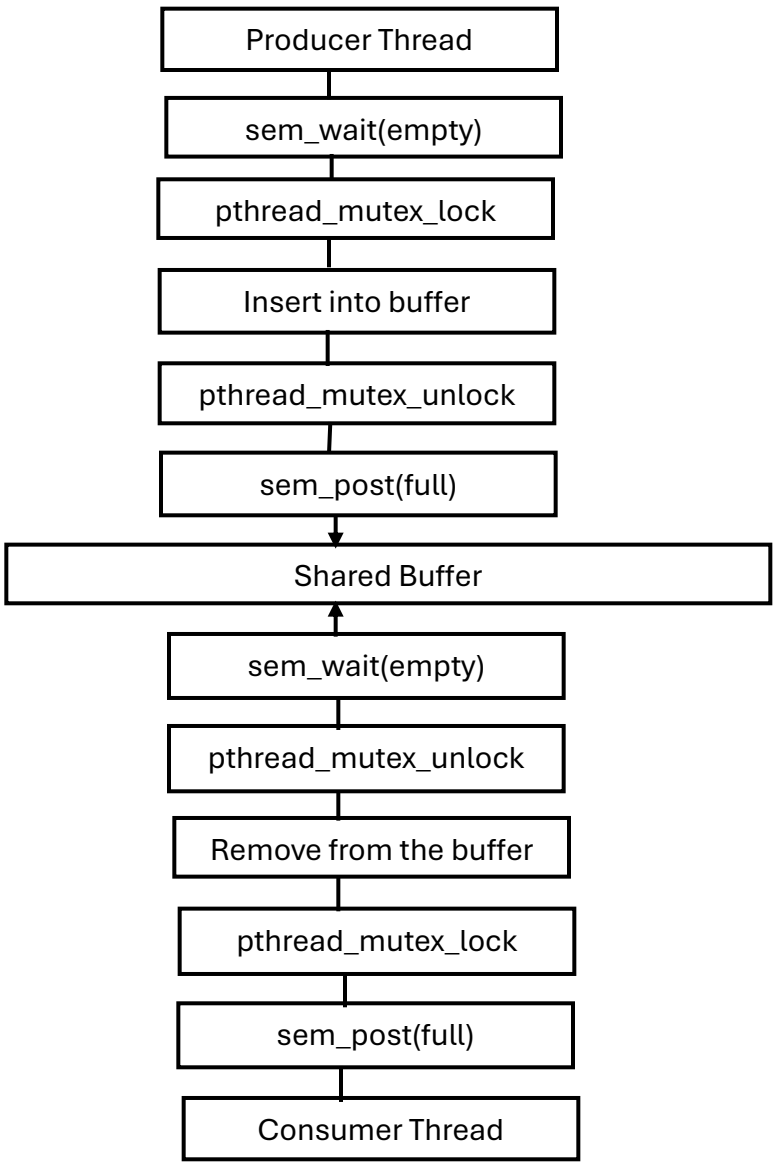
1. **Producer:**
   a. Wait for empty slots
   b. Lock mutex
   c. Insert item
   d. Unlock mutex
   e. Signal full

2. **Consumer:**
   a. Wait for available item
   b. Lock mutex
   c. Remove item
   d. Unlock mutex
   e. Signal empty

With this no buffer overflow, underflow, no race conditions and proper synchronization between threads.

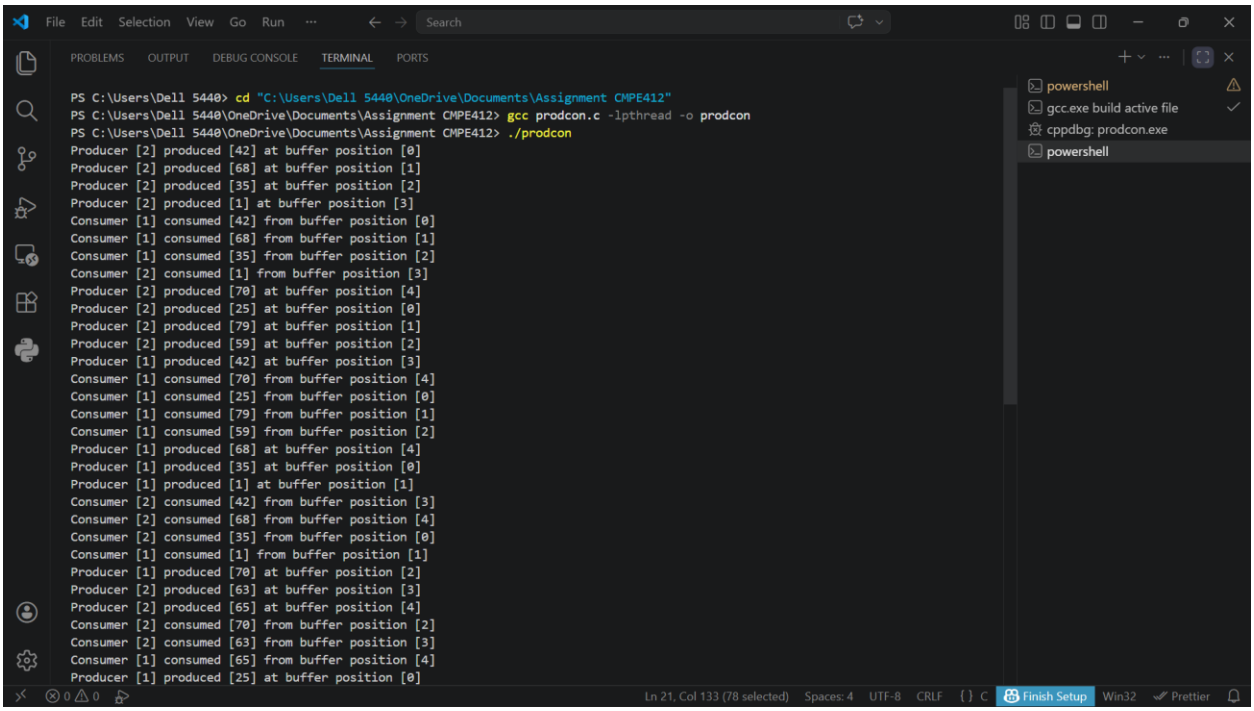
Producer_Consumer

Diagram:

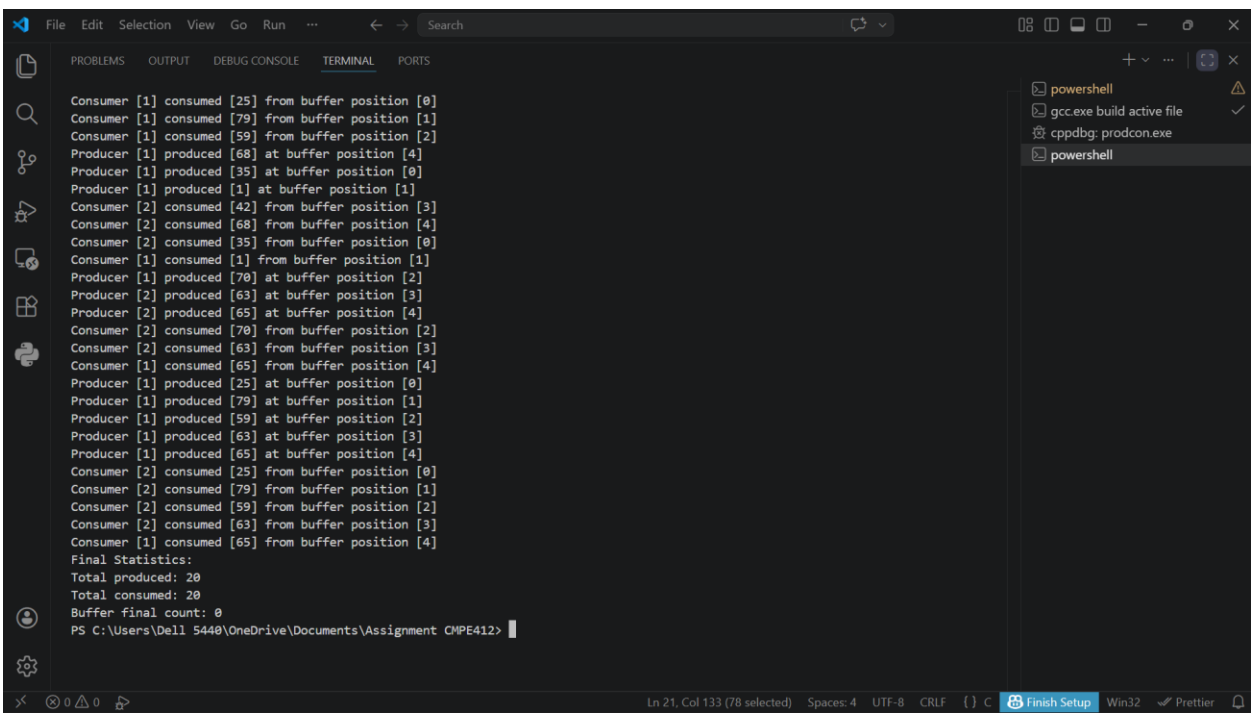


Producer_Consumer

Screenshot of Program Execution



```
PS C:\Users\Dell 5440> cd "C:\Users\Dell 5440\OneDrive\Documents\Assignment CMPE412"
PS C:\Users\Dell 5440\OneDrive\Documents\Assignment CMPE412> gcc prodcon.c -lpthread -o prodcon
PS C:\Users\Dell 5440\OneDrive\Documents\Assignment CMPE412> ./prodcon
Producer [2] produced [42] at buffer position [0]
Producer [2] produced [68] at buffer position [1]
Producer [2] produced [35] at buffer position [2]
Producer [2] produced [1] at buffer position [3]
Consumer [1] consumed [42] from buffer position [0]
Consumer [1] consumed [68] from buffer position [1]
Consumer [1] consumed [35] from buffer position [2]
Consumer [2] consumed [1] from buffer position [3]
Producer [2] produced [70] at buffer position [4]
Producer [2] produced [25] at buffer position [0]
Producer [2] produced [79] at buffer position [1]
Producer [2] produced [59] at buffer position [2]
Producer [1] produced [42] at buffer position [3]
Consumer [1] consumed [70] from buffer position [4]
Consumer [1] consumed [25] from buffer position [0]
Consumer [1] consumed [79] from buffer position [1]
Consumer [1] consumed [59] from buffer position [2]
Producer [1] produced [68] at buffer position [4]
Producer [1] produced [35] at buffer position [0]
Producer [1] produced [1] at buffer position [1]
Consumer [2] consumed [42] from buffer position [3]
Consumer [2] consumed [68] from buffer position [4]
Consumer [2] consumed [35] from buffer position [0]
Consumer [1] consumed [1] from buffer position [1]
Producer [1] produced [70] at buffer position [2]
Producer [2] produced [63] at buffer position [3]
Producer [2] produced [65] at buffer position [4]
Consumer [2] consumed [70] from buffer position [2]
Consumer [2] consumed [63] from buffer position [3]
Consumer [1] consumed [65] from buffer position [4]
Producer [1] produced [25] at buffer position [0]
```



```
Consumer [1] consumed [25] from buffer position [0]
Consumer [1] consumed [79] from buffer position [1]
Consumer [1] consumed [59] from buffer position [2]
Producer [1] produced [68] at buffer position [4]
Producer [1] produced [35] at buffer position [0]
Producer [1] produced [1] at buffer position [1]
Consumer [2] consumed [42] from buffer position [3]
Consumer [2] consumed [68] from buffer position [4]
Consumer [2] consumed [35] from buffer position [0]
Consumer [1] consumed [1] from buffer position [1]
Producer [1] produced [70] at buffer position [2]
Producer [2] produced [63] at buffer position [3]
Producer [2] produced [65] at buffer position [4]
Consumer [2] consumed [70] from buffer position [2]
Consumer [2] consumed [63] from buffer position [3]
Consumer [1] consumed [65] from buffer position [4]
Producer [1] produced [25] at buffer position [0]
Producer [1] produced [79] at buffer position [1]
Producer [1] produced [59] at buffer position [2]
Producer [1] produced [63] at buffer position [3]
Producer [1] produced [65] at buffer position [4]
Consumer [2] consumed [25] from buffer position [0]
Consumer [2] consumed [79] from buffer position [1]
Consumer [2] consumed [59] from buffer position [2]
Consumer [2] consumed [63] from buffer position [3]
Consumer [1] consumed [65] from buffer position [4]
Final Statistics:
Total produced: 20
Total consumed: 20
Buffer final count: 0
PS C:\Users\Dell 5440\OneDrive\Documents\Assignment CMPE412>
```

Producer_Consumer

Challenges Faced:

1. Had challenges with the is_buffer_empty and is_buffer_full check on consumer and producer functions as they were resulting in race conditions since they were placed inside the for loop but not on the shared memory but ended up removing the checks since the semaphore does this checks the empty and the full.
2. The buffer_insert duplication, I called it manually by writing tail++ and count++ on producer function
3. Had problems with the ThreadArgs and I ended up making the cbuffer as global for all the threads to access it

What happens if the mutex is removed

The mutex ensures mutual exclusion when accessing shared buffer variables, so if removed race conditions occurs where two producers will be accessing the shared buffer at the same time.

For example, both would read tail = 2, write buffer [2], and increment count meaning one item is overwritten and counts ends up as 3 instead of 4, just like the N++ race condition in Thread.pdf (Lecturer note reference_CIU) where one increment was lost